

中国科学技术大学

实验报告

课程名称: 计算机图形学

实验七: 光栅化与路径追踪

专业班级 2023 级计算机图形学

学 号 PB23000052

姓 名 杨硕

指导教师 刘利刚

日 期 2026 年 04 月 26 日

中国科学技术大学

目 录

1 实验概述	1
2 代码结构	1
3 基础任务	2
3.1 场景导入与光栅化节点	2
3.2 Blinn-Phong 着色与法线贴图	3
3.3 Shadow Mapping	4
3.4 路径追踪：矩形光源	5
3.5 路径追踪：递归着色与 Russian Roulette	7
4 可选任务	7
4.1 PCSS	7
4.2 SSAO	8
4.3 Displacement Mapping	9
4.4 更复杂 BRDF 与 MIS	9
5 拓展任务	10
5.1 光栅化与路径追踪对比	10
5.2 参数修改与效果对比	10
5.3 非真实感渲染	11
6 实验总结	11
7 AI 辅助说明	11

1 实验概述

本次实验围绕两条渲染路线展开：一部分基于光栅化管线，完成 G-Buffer、Blinn-Phong 着色、法线贴图、Shadow Mapping，以及可选的 PCSS、SSAO、Displacement Mapping；另一部分基于 Embree 路径追踪器，补全矩形光源的采样/相交/Irradiance 计算，并实现递归路径追踪、Russian Roulette、以及更复杂的 BRDF 与 MIS。

在开始撰写本报告前，我重新阅读了作业 PDF，并对 `hd_RUZINO_GL` 与 `hd_RUZINO_Embree` 两个插件进行了逐文件复核，不直接相信之前 AI 已修改过的版本。最终报告中的实现说明以当前代码状态为准，其中本轮进一步修正的重点包括：

- 路径追踪中递归项与直接光照的重复计数问题；
- 矩形光源与远光源的采样 pdf / 交点计算一致性；
- 光栅化法线贴图中 TBN 构造的稳健性。

2 代码结构

本次实验主要涉及如下文件：

- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/node_render_rasterize_gl.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/node_render_shadow_mapping_gl.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/node_render_deferred_lighting_gl.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/node_render_ssao_gl.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/shaders/rasterize_impl.vs
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/shaders/rasterize_impl.fs

- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/shaders/
blinn_phong.fs
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/shaders/
ssao.fs
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/shaders/
rasterize_displacement.vs
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL/nodes/shaders/
toon_lighting.fs
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_Embree/renderer.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_Embree/integrator.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_Embree/integrators/
path.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_Embree/material.cpp
- Framework3D/Ruzino/source/Plugins/hd_RUZINO_Embree/light.cpp
- Framework3D/Ruzino/source/Plugins/
hd_RUZINO_Embree/surfaceInteraction.h

3 基础任务

3.1 场景导入与光栅化节点

光栅化部分沿用作业文档推荐的节点式流程：先用 `rasterize_impl` 输出位置、深度、法线、材质参数等 G-Buffer，再用 `shadow_mapping` 生成阴影图，最后在 `deferred_lighting` 中进行 Blinn-Phong 着色与阴影计算。

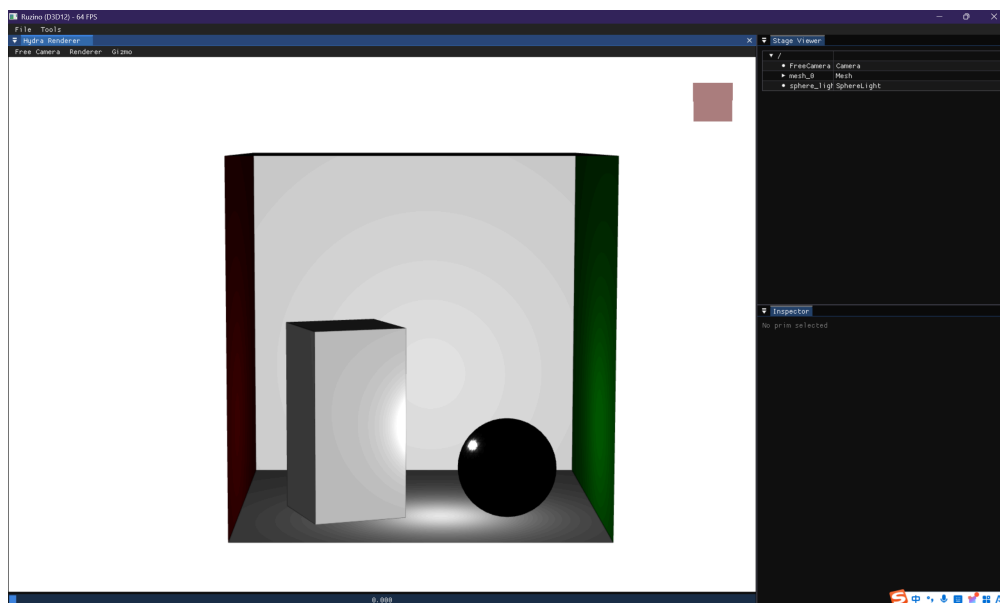


图 3-1 光栅化场景导入示意。

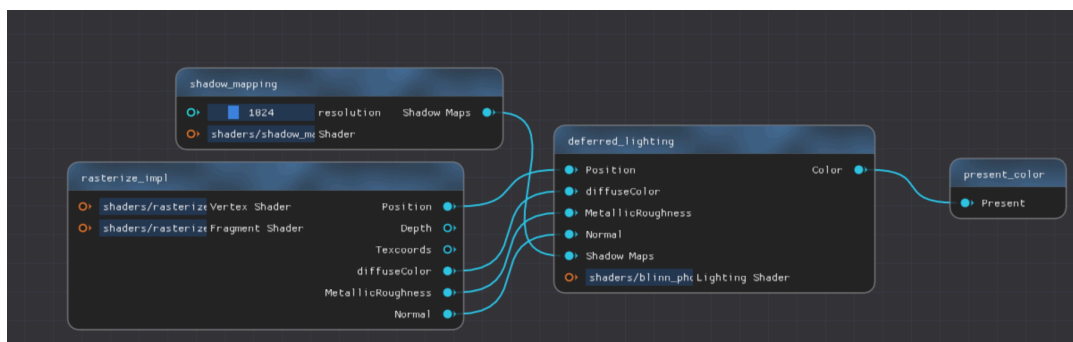


图 3-2 基础光栅化节点连接方式。rasterize_impl 负责生成 G-Buffer, shadow_mapping 负责从光源视角生成深度图, deferred_lighting 则完成延迟着色。

3.2 Blinn-Phong 着色与法线贴图

rasterize_impl.fs 中首先输出世界空间位置、深度、纹理坐标、漫反射颜色、金属度/粗糙度与法线。法线贴图部分并不是简单把切线空间法线直接当世界空间法线使用,而是先根据 $dFdx/dFdy$ 得到位置和 UV 的局部变化率,再构造 TBN 矩阵,把法线贴图采样值从切线空间变换到世界空间。

相比于只用单一叉乘构造切线,本次实现额外处理了两类数值问题:

- 当 UV 雅可比行列式接近 0 时,使用几何法线与局部导数构造退化 fallback,避免切线为零;
- 将最终扰动法线限制在几何法线同一半球内,避免地面等大平面出现法线翻转。

在延迟着色阶段,blinn_phong.fs 读取 G-Buffer,按如下方式计算颜色:

实验七：光栅化与路径追踪

- 环境项: $\text{ambient} = 0.03 * \text{albedo}$
- 漫反射项: $\text{diffuse} = \text{albedo} * \max(\text{dot}(\mathbf{n}, \mathbf{l}), 0)$
- 高光项: 采用 Blinn-Phong 半程向量 $\mathbf{h} = \text{normalize}(\mathbf{l} + \mathbf{v})$, 并以粗糙度控制 shininess
- 金属 workflows: 用 $\text{mix}(\text{vec3}(0.04), \text{albedo}, \text{metal})$ 作为高光颜色

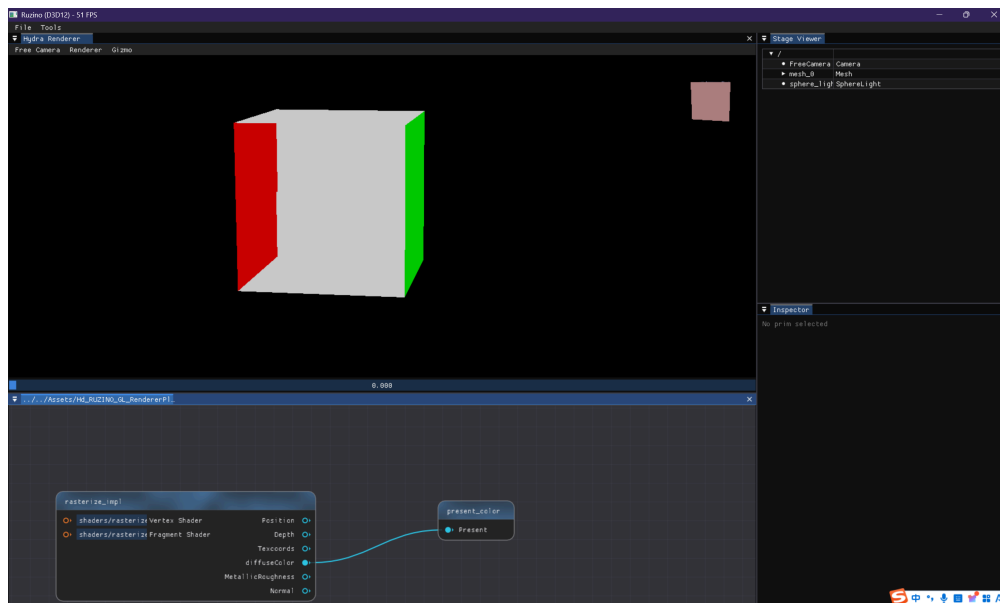


图 3-3 G-Buffer 中的漫反射颜色结果。

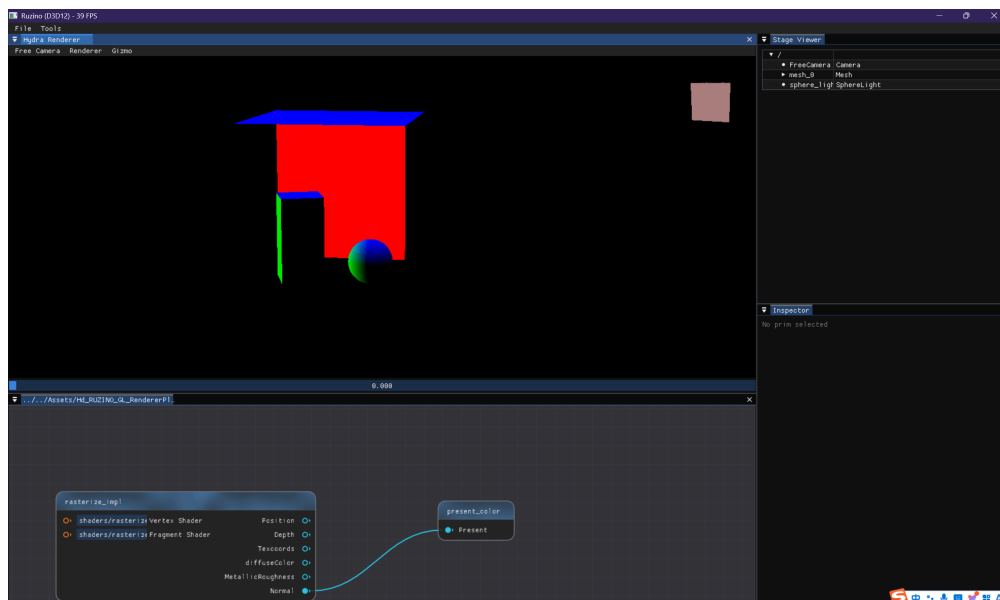


图 3-4 G-Buffer 中的法线缓冲结果。正确的法线贴图应能在砖块、墙面等区域形成稳定的局部扰动。

3.3 Shadow Mapping

Shadow Mapping 的实现分为两步:

1. `node_render_shadow_mapping_gl.cpp` 遍历场景光源，从球光源视角构造 `light_view` 与 `light_projection`，并将各个模型渲染到阴影图数组中；
2. `node_render_deferred_lighting_gl.cpp` 将光源位置、颜色、投影矩阵、视图矩阵和阴影图层编号打包后传给 `blinn_phong.fs`，再在延迟着色中完成光空间投影、深度比较与 `bias` 修正。

由于阴影图按照光源逐层存入 `sampler2DArray`，因此一个着色 `pass` 即可支持多光源阴影查询。片元着色时先将世界空间位置投影到光源裁剪空间，再换算为 `shadow_uv`，最后比较当前深度与阴影图深度决定是否被遮挡。

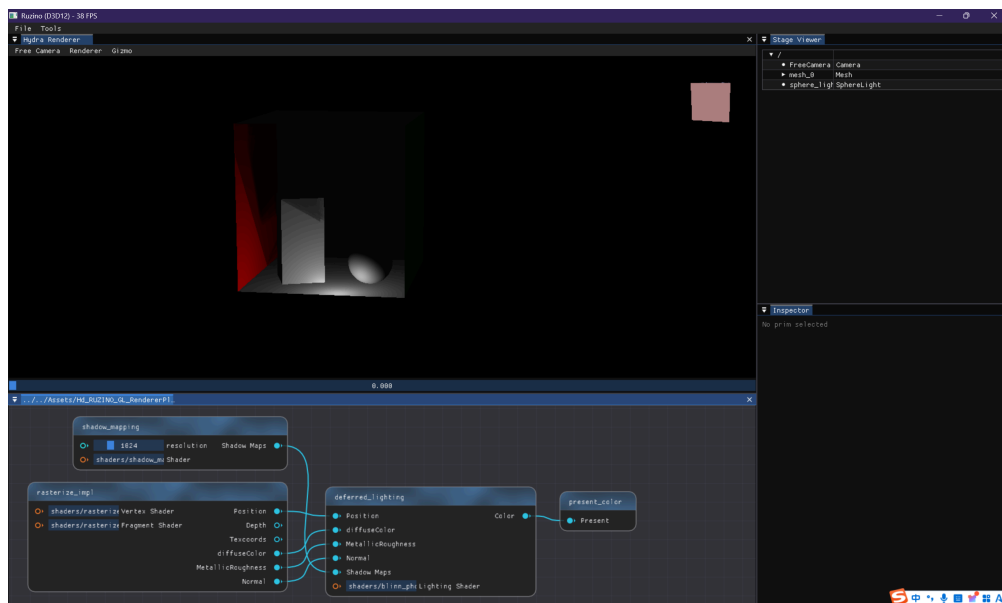


图 3-5 开启 Shadow Mapping 后的结果。物体与地面接触处出现清晰阴影，说明光空间投影和深度比较已经接通。

3.4 路径追踪：矩形光源

矩形光源的实现主要在 `hd_RUZINO_Embree/light.cpp` 中完成，包括三部分：

实验七：光栅化与路径追踪

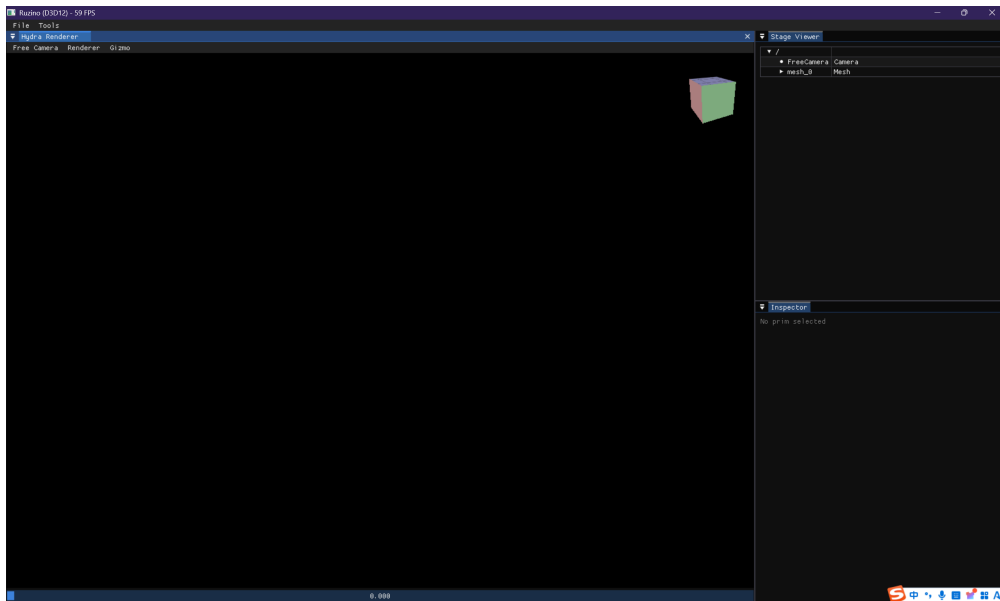


图 3-6 路径追踪场景导入示意。实验中可使用 Cornell Box 与矩形光源检查采样、相交和直接光照效果。

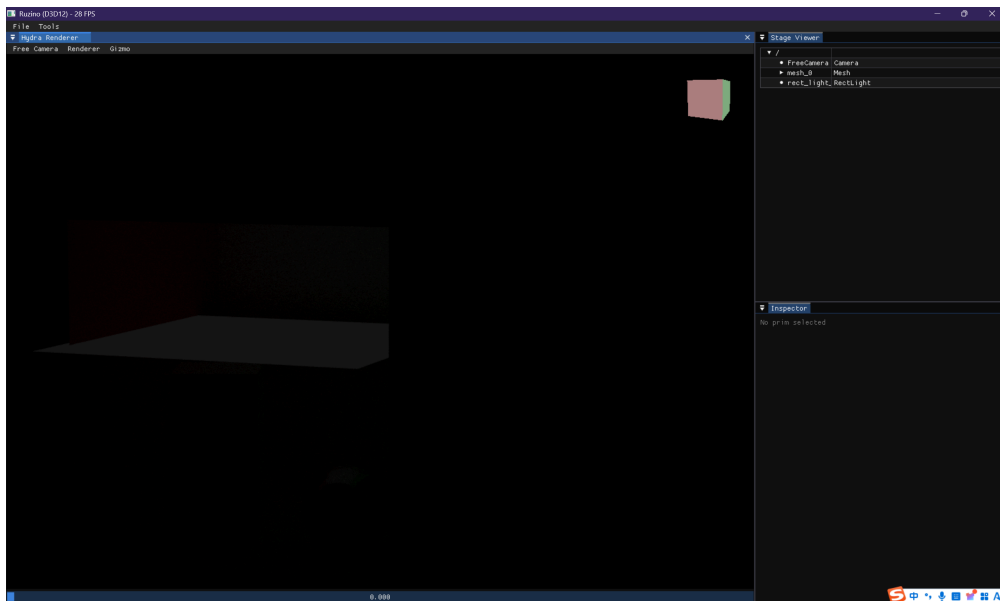


图 3-7 路径追踪结果示意。图中使用了矩形光源，能够观察到由面光源带来的照明分布与阴影效果，可用于验证矩形光源采样、相交和 Irradiance 计算已经接入路径追踪流程。

- 采样：在由 `corner0`、`corner1`、`corner2` 定义的矩形面上均匀采样点，再通过面积测度到立体角测度的变换得到 $\text{pdf} = \text{distance}^2 / (\cos(\theta) * \text{area})$ ；
- 相交：先求射线与矩形所在平面的交点，再在矩形局部坐标中解出参数 (u, v) ，判断交点是否落在 $[0, 1]^2$ 范围内；
- Irradiance：根据实际世界空间面积 $|\text{cross}(\text{edge}_u, \text{edge}_v)|$ 计算面积，再由 $\text{irradiance} = \text{power} / \text{area}$ 得到单位面积出射能量。

这里我额外修正了两个细节：

- 光源面积不能只用局部 `width * height`，而应使用变换后的世界空间边向量叉积长度；
- 若递归射线直接命中矩形光源，其返回的辐射亮度必须和 `PdfLi` 保持一致，才能正确参与 MIS。

3.5 路径追踪：递归着色与 Russian Roulette

路径追踪器通过 `renderer.cpp` 切换到 `PathIntegrator`。在 `integrators/path.cpp` 中，单条路径的估计由两部分组成：

- 直接光照：调用 `EstimateDirectLight(si, uniform_float)` 完成 next-event estimation；
- 间接光照：从材质 BRDF 中采样一个新方向，递归估计入射辐射亮度，再乘以 BRDF、余弦项和 pdf 反比。

Russian Roulette 的策略为：

- 前 3 次弹射不裁剪，终止概率为 0；
- 之后使用 `rr_prob = 0.8`；
- 若路径存活，则递归项除以 `rr_prob` 保持无偏。

在重新检查已有代码后，我额外修正了一个重要问题：若当前像素已经通过 MIS 估计过直接光照，那么递归弹出的 BRDF 射线若再次直接命中光源，就会造成“直接光照被算两次”。因此当前版本中只允许主光线直接返回光源/环境辐射；对于递归射线，直接光源贡献统一交给 `EstimateDirectLight` 处理，从而避免重复计数。

4 可选任务

4.1 PCSS

`blinn_phong.fs` 中在基础 Shadow Mapping 之上实现了 blocker search + PCF filter 的近似 PCSS。核心步骤是：

- 先在小邻域中搜索 blocker 深度并求平均；
- 以当前片元深度与 blocker 平均深度的差估计半影宽度；
- 用估计出的过滤半径执行可变半径 PCF。

实验七：光栅化与路径追踪

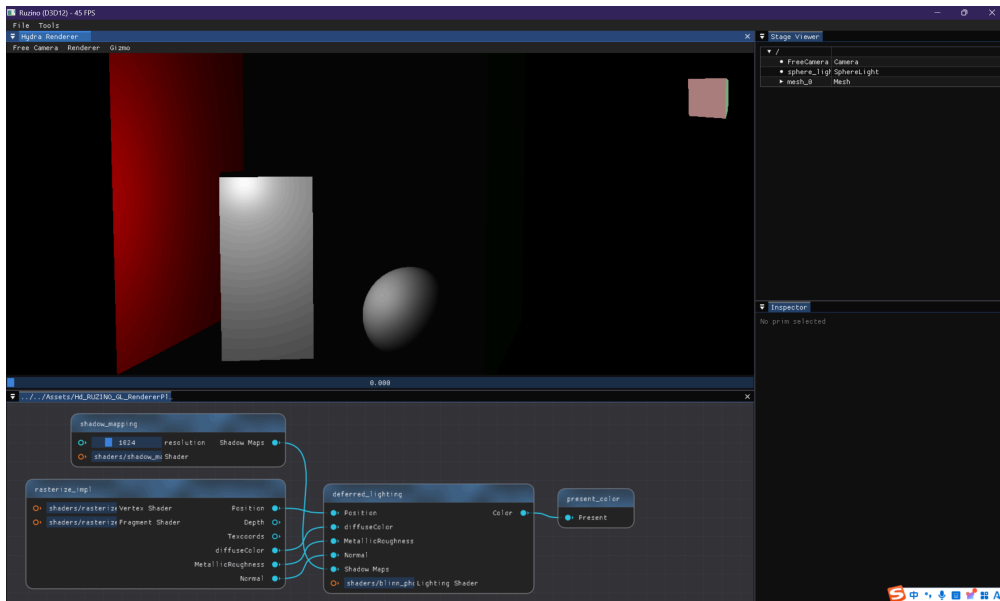


图 4-1 PCSS 结果。阴影边缘由硬阴影变为具有一定宽度的软阴影。

4.2 SSAO

SSAO 在 `node_render_ssao_gl.cpp` 与 `ssao.fs` 中实现。该 pass 读取 Color、Position、Depth 与 Normal，在屏幕空间中围绕当前像素进行多次环形采样，并依据深度差、法线夹角与距离衰减估计环境遮蔽因子，最后将 AO 结果乘回原始颜色。

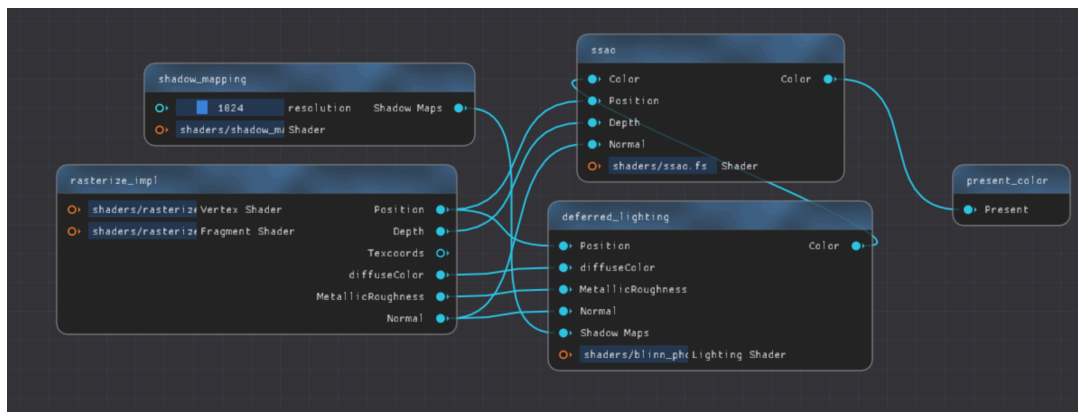


图 4-2 SSAO 节点连接方式。通常将其接在 deferred_lighting 之后。

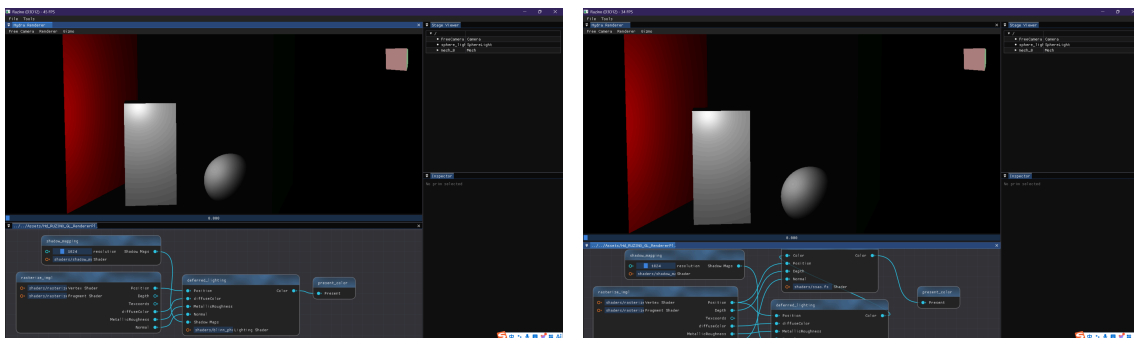


图 4-3 SSAO 关闭（左）与开启（右）的对比。开启后角落、接触边与缝隙区域具有更明显的遮蔽感。

4.3 Displacement Mapping

位移贴图通过单独的顶点着色器 `rasterize_displacement.vs` 完成。该着色器在模型空间读取位移纹理，将 `aPos` 沿顶点法线方向偏移，再进入后续的光栅化和延迟着色流程。由于该过程直接修改几何位置，因此相较于法线贴图会同时改变轮廓与自阴影关系。

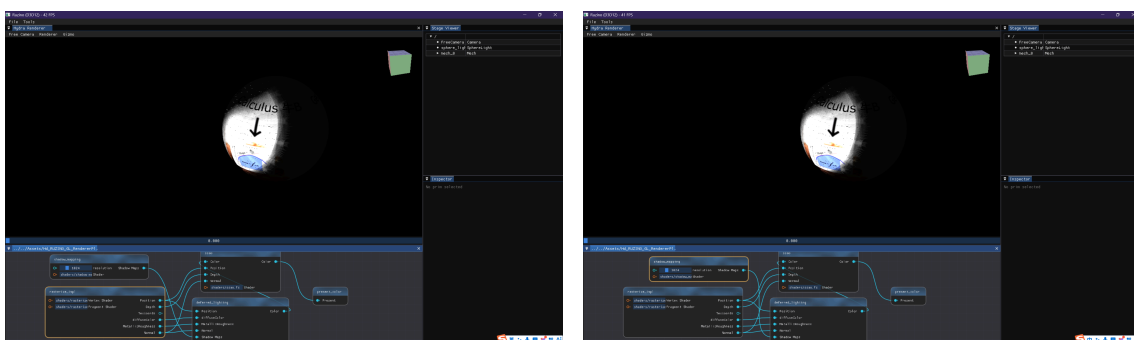


图 4-4 Displacement Mapping 关闭（左）与开启（右）的对比。开启后表面几何起伏更加明显。

4.4 更复杂 BRDF 与 MIS

路径追踪材质不再使用简单 Lambert 模型，而是在 `material.cpp` 中实现了基于 GGX 的微表面 BRDF：

- 法线分布函数：GGX
- 菲涅耳项：Schlick Fresnel
- 几何遮蔽：Smith Geometry
- 漫反射： $kd * \text{diffuse} / \pi$

在 `integrator.cpp` 中，直接光照使用了双重采样与 Power Heuristic MIS：

- 从光源分布采样一条方向，得到 light-sampled contribution；
- 从 BRDF 采样一条方向，若直接命中光源，则得到 brdf-sampled contribution；

- 最后按 $w = f^2 / (f^2 + g^2)$ 进行加权。

这使得高光材质与面光源同时存在时，估计方差明显低于只做单侧采样的版本。

5 拓展任务

5.1 光栅化与路径追踪对比

两条渲染路线的特点如下：

- 光栅化：依赖显式 G-Buffer 和局部着色，交互性强、实时性好，适合快速调参和观察法线/深度等中间缓冲；
- 路径追踪：能够自然表达全局光照、软阴影、间接反射与复杂 BRDF，但采样数不足时噪声更大，耗时也明显更高。

在当前实现中，适合对比的典型项目包括：

- 光栅化 Shadow Mapping / PCSS 与路径追踪面光源软阴影的差异；
- 光栅化局部 Blinn-Phong 与路径追踪 GGX+MIS 在高光形状上的差异；
- 相同场景下实时预览速度与路径追踪收敛速度的差异。

由于本次报告中保存的截图以功能验证为主，这一部分主要给出文字层面的对比总结。就当前实现而言，光栅化更适合实时交互和快速调参，便于观察 G-Buffer、阴影图与屏幕空间效果；路径追踪虽然计算代价更高，但在矩形面光源、间接光照和复杂高光响应上更自然，也更符合真实光照传播过程。

5.2 参数修改与效果对比

本次实验中可以直接调节并进行对比的参数包括：

- 光栅化：Shadow Mapping 分辨率、PCSS 搜索半径、PCF 核大小、SSAO 半径、SSAO sample count、Displacement scale；
- 路径追踪：SPP、Russian Roulette 起始深度、RR 保留概率、粗糙度、金属度、矩形光源宽高与强度。

这些参数分别影响阴影锐度、软阴影宽度、遮蔽强度、表面起伏、噪声水平和材质外观。实验上建议固定场景后只变化单一参数，以便观察对应的视觉变化来源。

结合本次已有截图，可以直接观察到如下趋势：PCSS 相较于基础 Shadow Mapping 会让阴影边缘更柔和；SSAO 会加强接触区域和角落处的遮蔽感；

Displacement Mapping 会显著增强表面几何层次；而路径追踪结果则表明，矩形面光源能够产生更加平滑自然的明暗过渡。

5.3 非真实感渲染

除了真实感着色外，本次代码中还提供了 `toon_lighting.fs` 作为非真实感渲染拓展。该 shader 采用以下策略：

- 对漫反射 $n \cdot l$ 做分段量化，形成卡通色阶；
- 用 `step` 函数量化高光；
- 继续复用阴影图，将阴影也纳入卡通渲染风格。

由于本次提交中没有额外保存对应效果图，本报告对这一拓展项主要记录实现思路而不展开图像对比。它说明当前节点框架不仅支持真实感延迟着色，也适合在相同 G-Buffer 基础上快速替换为 NPR 风格着色器。

6 实验总结

本次实验分别从光栅化与路径追踪两条路线实现了完整的渲染功能链。光栅化部分完成了 G-Buffer、法线贴图、Blinn-Phong、Shadow Mapping，并进一步扩展到 PCSS、SSAO 和 Displacement Mapping；路径追踪部分补全了矩形光源、递归着色、Russian Roulette、以及基于 GGX 的 BRDF 和 MIS。在重新审核已有代码后，我重点修复了法线贴图稳定性、矩形光源面积/采样一致性，以及路径追踪中直接光照重复计数的问题，使当前版本更接近作业要求下“既能出图、又有理论一致性”的状态。

7 AI 辅助说明

本次实验使用 Codex 作为编程和文档辅助工具，但整体流程不是“直接接受 AI 生成结果”，而是多轮对话、复核与修正。下面按交互顺序概述本次 AI 辅助过程。

首先，我要求 AI 阅读 hw7 的 README 与 `rtfd-hw7.pdf`，并明确指出不能相信仓库里已经被 AI 改过一遍的代码，需要重新审查基础任务、可选任务和拓展任务是否真正完成。AI 随后先检查了 `Homeworks/7_rasterization_and_path_tracing` 的目录结构，再把注意力集中到 `Framework3D/Ruzino/source/Plugins/hd_RUZINO_GL` 与 `hd_RUZINO_Embree` 两个渲染插件。

实验七：光栅化与路径追踪

接着，为了满足“必须打开 PDF”的要求，AI 尝试使用本地 `pdftotext` 提取文档内容，但因为 MiKTeX 首次初始化权限问题失败；之后改为安装轻量级 `pypdf`，成功读取了 PDF 的关键页，并据此确认本次作业要求包括 Blinn-Phong、法线贴图、Shadow Mapping、矩形光源、路径追踪递归与 Russian Roulette，以及 PCSS、SSAO、Displacement Mapping、复杂 BRDF、MIS 和非真实感渲染等扩展项。

在代码复核阶段，AI 没有只搜索 TODO，而是逐个阅读了关键 shader 与积分器实现。例如在光栅化部分，它检查了 `rasterize_impl.fs`、`blinn_phong.fs`、`node_render_shadow_mapping_gl.cpp`、`node_render_deferred_lighting_gl.cpp` 和 `ssao.fs`，判断法线贴图、延迟着色、阴影图生成、PCSS 与 SSAO 是否已经接通；在路径追踪部分，它检查了 `renderer.cpp`、`integrator.cpp`、`path.cpp`、`material.cpp`、`light.cpp` 和 `surfaceInteraction.h`，确认渲染器是否切换到 `PathIntegrator`、矩形光源是否真正实现、BRDF 与 MIS 是否一致，以及递归项是否无偏。

在这一轮复核后，AI 指出了几个“不应直接信任现状”的点。第一，法线贴图虽然已有实现，但 TBN 构造在 UV 退化和半球翻转情况下不够稳健；第二，矩形光源采样、相交和 `pdfLi` 虽然已有部分代码，但面积与世界空间变换的一致性需要核查；第三，路径追踪递归项虽然已经写出框架，但递归射线直接命中光源时，可能与显式直接光照采样产生重复计数。

随后，AI 对上述问题进行了补改。在 `rasterize_impl.fs` 中，重写了切线与副切线的构造逻辑，增加 UV 退化时的 `fallback`，并将扰动法线约束在几何法线同半球内。在 `light.cpp` 中，修正了远光源的采样与 `pdf`，完善了矩形光源的表面采样、平面相交、局部坐标判断、面积到立体角测度的 `pdf` 变换，并用世界空间边向量叉积计算面积与 Irradiance。在 `path.cpp` 中，补全了 Russian Roulette，并修正了递归路径中“递归射线命中光源”和“显式直接光照估计”重复计算的问题。

在验证阶段，AI 原本尝试用命令行直接调用 CMake 构建以检查语法和链接问题，但我随后明确要求不要继续编译，因为该工程只能通过 Visual Studio 进行构建。收到这一约束后，AI 停止了命令行编译，改为仅进行静态代码审查、`git diff` 核对和文档整理，不再尝试执行构建。

实验七：光栅化与路径追踪

进入报告阶段后，我要求 AI 直接撰写 hw7 的实验报告，并允许使用 `report/images` 中已有截图。AI 先通过 `git status` 与 `git diff` 对比当前工作区，区分出“仓库中此前已改动的 hw7 相关文件”和“本轮新增修正的文件”，再据此重写 `report/main.typ` 和 `report/appendix.typ`，将基础任务、可选任务、拓展任务、代码结构、实现细节与 AI 辅助说明统一整理为 hw7 对应版本。

最后，在排版修复阶段，我继续指出报告中的细节问题，例如 Typst 中双图 `figure` 写法不合法、封面课程名分行、页眉文字过长、AI 辅助说明过于简略等。AI 根据这些反馈继续修改报告：把双图改为 `grid(...)` 包裹、将封面和页眉收缩为简短中文形式、移除“建议补充截图”这类不应出现在正式报告中的内容，并把 AI 辅助过程扩写为本节所示的对话摘要。

总体而言，本次 AI 主要承担了四类辅助工作：一是阅读作业 PDF 和定位代码路径；二是检查现有实现是否真的满足要求，而不是只看是否“已经写过”；三是协助补改少量关键 bug 与一致性问题；四是整理实验报告结构、把实现与任务条目建立明确对应关系。最终提交内容仍以人工确认后的代码与文档为准。